Especificação Técnica – Plataforma Ponto de Apoio

1. Contextualização Social e Escopo Extensionista

A Ponto de Apoio é uma ONG fictícia, sem fins lucrativos, localizada em Itajaí, Santa Catarina. Seu objetivo é ajudar famílias e pessoas em situação de vulnerabilidade social, realizando campanhas de arrecadação de alimentos, roupas, produtos de higiene e outras ações comunitárias.

Atualmente, a ONG enfrenta dificuldades para organizar suas campanhas, registrar as doações recebidas e controlar a participação dos voluntários. Muitas dessas informações são compartilhadas por mensagens ou anotadas manualmente, o que pode causar confusões e dificultar o acompanhamento das atividades.

Pensando nisso, o projeto propõe a criação de uma plataforma web chamada **Ponto de Apoio**, que terá como objetivo reunir as principais informações da ONG em um único lugar, facilitando a organização das campanhas e a participação da comunidade.

1.1. Identificação da entidade

Nome: Ponto de Apoio

Natureza: Organização não governamental fictícia, sem fins lucrativos.

Localização: Itajaí – Santa Catarina.

Área de atuação: Assistência social e apoio comunitário.

Público atendido: Famílias e pessoas em situação de vulnerabilidade social na cidade de Itajaí.

Principais atividades: Arrecadação de alimentos, roupas, produtos de higiene, organização de campanhas solidárias e ações de voluntariado.

1.2. Problema identificado

A ONG Ponto de Apoio precisa organizar melhor suas campanhas e as informações relacionadas às doações e aos voluntários.

Entre os principais problemas identificados no cenário proposto estão:

- **Controle das doações:** dificuldade para registrar quem realizou uma doação, o que foi doado e para qual campanha.
- **Divulgação das campanhas:** as pessoas nem sempre conseguem encontrar informações sobre campanhas ativas e suas necessidades.
- **Organização dos voluntários:** dificuldade para acompanhar quem deseja participar de uma ação e quantas vagas ainda estão disponíveis.
- **Acompanhamento das metas:** falta de um local onde os responsáveis e a comunidade possam consultar o andamento das campanhas.

Essas dificuldades podem atrapalhar a organização das atividades e diminuir a participação da comunidade nas ações sociais.

1.3. Justificativa

A criação da plataforma Ponto de Apoio busca facilitar a organização das atividades da ONG e aproximar a comunidade das campanhas solidárias.

Com o sistema, os usuários poderão consultar campanhas disponíveis, conhecer suas necessidades e registrar doações ou inscrições para participar como voluntários.

Já os responsáveis pela ONG poderão cadastrar novas campanhas e acompanhar as doações e os participantes de cada ação.

Além de ajudar na organização, a plataforma poderá dar mais visibilidade às campanhas e facilitar o acesso às informações, incentivando a participação da comunidade.

O projeto também permitirá aplicar os conhecimentos desenvolvidos na disciplina de Programação Web, utilizando Vue.js para a interface e Node.js com Express.js para o servidor.

2. Perfis de Usuários

A plataforma Ponto de Apoio terá três tipos principais de usuários: doadores, voluntários e administradores da ONG.

Cada um terá funções diferentes dentro do sistema, de acordo com sua participação nas campanhas e ações sociais.

Nesta primeira versão, não será necessário criar uma conta ou realizar login. Os usuários poderão acessar as campanhas e escolher a ação que desejam realizar. Essa decisão foi tomada para deixar o sistema mais simples durante o desenvolvimento inicial.

2.1. Doador

O doador será qualquer pessoa ou empresa que deseja contribuir com as campanhas realizadas pela ONG.

Principais funções:

- Visualizar as campanhas disponíveis.
- Consultar informações e necessidades de cada campanha.
- Registrar doações em dinheiro ou itens, como alimentos, roupas e produtos de higiene.
- Acompanhar o progresso das campanhas e suas metas.

2.2. Voluntário

O voluntário será a pessoa interessada em dedicar seu tempo para ajudar nas atividades organizadas pela ONG.

Principais funções:

- Visualizar as campanhas e ações que precisam de voluntários.
- Consultar as informações de cada atividade.
- Verificar a disponibilidade de vagas.
- Realizar sua inscrição como voluntário.
- Informar seus dados de contato e disponibilidade.

2.3. Administrador da ONG

O administrador será o responsável pela organização das campanhas e pelo acompanhamento das atividades da Ponto de Apoio.

Principais funções:

- Cadastrar novas campanhas.
- Informar os objetivos, as metas e as necessidades de cada campanha.
- Acompanhar as doações registradas.
- Consultar os voluntários inscritos.
- Verificar o andamento das campanhas e a quantidade de vagas disponíveis.

2.4. Resumo dos Perfis

Perfil	**Quem é**	**O que faz no sistema**
Doador	Pessoa ou empresa que deseja ajudar	Consulta campanhas e registra doações
Voluntário	Pessoa interessada em participar das ações sociais	Consulta atividades e se inscreve como voluntário
Administrador	Responsável pela organização da ONG	Cadastra campanhas e acompanha doações e voluntários

3. Requisitos e Regras de Negócio

Nesta etapa, serão definidos os principais requisitos e regras de funcionamento da plataforma Ponto de Apoio.

Os requisitos funcionais descrevem as ações que os usuários poderão realizar. Os requisitos não funcionais definem características técnicas do sistema, enquanto as regras de negócio estabelecem as condições que deverão ser respeitadas durante sua utilização.

3.1. Requisitos Funcionais

Os requisitos funcionais representam as principais funcionalidades que a plataforma deverá oferecer aos usuários.

Código	Requisito	Usuário
RF01	Exibir as campanhas cadastradas, mostrando nome, categoria, situação e progresso da arrecadação.	Todos
RF02	Permitir filtrar as campanhas por situação (ativa ou encerrada).	Todos
RF03	Permitir visualizar os detalhes de uma campanha, como descrição, objetivo, meta e informações sobre a arrecadação.	Todos
RF04	Exibir as doações registradas em cada campanha.	Todos
RF05	Permitir registrar doações em dinheiro ou itens, como alimentos, roupas e produtos de higiene.	Doador
RF06	Permitir que pessoas interessadas se inscrevam como voluntárias nas campanhas.	Voluntário
RF07	Permitir cadastrar novas campanhas, informando título, descrição, categoria, meta e quantidade de vagas para voluntários.	Administrador
RF08	Exibir mensagens de carregamento, confirmação e erro durante a utilização do sistema.	Todos

3.2. Requisitos Não Funcionais

Os requisitos não funcionais definem como o sistema deverá funcionar em relação à tecnologia, à comunicação entre suas partes e à facilidade de utilização.

Código	Requisito
RNF01	O frontend deverá ser desenvolvido em Vue.js 3, utilizando Vue Router para navegar entre as páginas sem precisar recarregar o site inteiro.
RNF02	O backend deverá utilizar Node.js com Express.js, disponibilizando uma API REST para comunicação com o frontend.
RNF03	Inicialmente, os dados serão armazenados em arrays de objetos na memória, sem a necessidade de um banco de dados.
RNF04	O sistema deverá permitir a comunicação entre frontend e backend por meio do CORS, quando estiverem funcionando em portas diferentes.
RNF05	A API deverá utilizar respostas em JSON e códigos HTTP adequados, como 200, 201, 400, 404, 409 e 500.

3.3. Regras de Negócio

As regras de negócio definem as condições necessárias para que as ações sejam realizadas corretamente dentro da plataforma.

Código	Regra de negócio
RN01	Apenas campanhas ativas poderão receber novas doações e inscrições de voluntários.
RN02	Para registrar uma doação em dinheiro, o valor informado deverá ser maior que zero e será acrescentado ao total arrecadado da campanha.
RN03	Para realizar uma doação de itens, será necessário informar a descrição do que está sendo doado. Esse tipo de doação não aumentará o valor arrecadado em dinheiro.
RN04	Para registrar uma doação ou inscrição, o usuário deverá informar um nome com pelo menos três caracteres e um e-mail válido.
RN05	O sistema identificará os doadores pelo e-mail informado. Caso o e-mail já esteja cadastrado, os dados existentes poderão ser reutilizados.
RN06	Cada inscrição de voluntário ocupará uma vaga disponível na campanha. Quando todas as vagas forem preenchidas, novas inscrições não serão aceitas.
RN07	Um voluntário não poderá se inscrever duas vezes na mesma campanha utilizando o mesmo e-mail.
RN08	Para cadastrar uma campanha, será obrigatório informar um título com pelo menos três caracteres, uma descrição e uma meta maior que zero. A campanha será criada como ativa, com o valor arrecadado inicialmente igual a zero.
RN09	Caso o usuário tente acessar uma campanha que não existe, o sistema deverá apresentar uma mensagem informando que ela não foi encontrada.

3.4. Funcionamento das Regras

Para entender melhor como essas regras serão aplicadas, podemos considerar algumas situações:

Exemplo 1 – Registro de doação

Uma pessoa deseja contribuir com R$ 50,00 para uma campanha de arrecadação de alimentos.

Ao informar seus dados e confirmar a doação, o sistema deverá verificar se a campanha está ativa e se o valor informado é válido. Caso esteja tudo correto, a doação será registrada e os R$ 50,00 serão adicionados ao total arrecadado da campanha.

Exemplo 2 – Inscrição de voluntário

Uma campanha precisa de cinco voluntários para ajudar na separação de alimentos.

Quando uma pessoa realiza sua inscrição, o sistema registra seus dados e diminui a quantidade de vagas disponíveis. Quando as cinco vagas forem preenchidas, novas inscrições não serão

permitidas.

Exemplo 3 – Cadastro de campanha

O responsável pela ONG deseja criar uma campanha para arrecadar roupas de inverno.

Ele deverá informar o nome da campanha, uma descrição, a categoria e a meta de arrecadação.

Após o cadastro, a campanha será adicionada à plataforma como ativa e poderá ser visualizada pelos demais usuários.

Exemplo 4 – Inscrição duplicada

Uma pessoa que já está inscrita como voluntária em determinada campanha tenta realizar outra inscrição utilizando o mesmo e-mail.

Nesse caso, o sistema deverá impedir o cadastro duplicado e apresentar uma mensagem explicando que essa pessoa já está inscrita na campanha.

4. Especificação da Interface e Navegação (Vue.js)

O frontend da Ponto de Apoio é uma *Single Page Application* (SPA) construída com Vue.js 3 (Composition API, `<script setup>`), empacotada pelo Vite e executada em `http://localhost:5173`. A navegação entre as telas é controlada pelo Vue Router, que troca apenas o conteúdo central da página, sem recarregar o site (RNF01).

4.1. Estrutura geral da aplicação

O componente raiz `App.vue` define o layout fixo da aplicação: o cabeçalho e o rodapé permanecem sempre visíveis e o componente `<RouterView />` é substituído pela tela correspondente à rota atual.

```
<!-- src/App.vue -->
<template>
  <HeaderComponent/>   <!-- logo + menu (Início | Cadastrar Campanha) -->
  <RouterView />       <!-- tela da rota atual: HomeView, CampaignView ou RegisterView -->
  <FooterComponent/>
</template>
```

A URL base da API é lida da variável de ambiente `VITE_API_URL` (arquivo `frontend/.env`), com valor `http://localhost:3000/api`. Todas as telas fazem as requisições com a função nativa `fetch`.

4.2. Mapa de rotas (Vue Router)

As rotas estão declaradas em `src/router/index.js`. O roteador usa o modo `createWebHistory` (URLs sem `#`) e cada tela é carregada sob demanda (*lazy loading*) por meio de `() => import(...)`.

Rota (path)	Nome	Componente	Parâmetros	Descrição
`/`	home	HomeView.vue	—	**Tela 1** – Visão geral: lista todas as campanhas com categoria, situação, progresso da meta e vagas de voluntariado.
`/campanha/:id`	campaign	CampaignView.vue	`:id` (parâmetro dinâmico de rota) – identificador numérico da campanha	**Tela 2** – Detalhes e acompanhamento de uma campanha: descrição, meta, valor arrecadado, período, doações e voluntários inscritos.

`/cadastrar` `/cadastrar?campanha=:id`	register	RegisterView.vue	`campanha` (query string, opcional)	**Tela 3** – Ação/Cadastro: formulário para doar, inscrever-se como voluntário ou cadastrar uma nova campanha.

```
// src/router/index.js
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/',             name: 'home',     component: () =>
import('../views/HomeView.vue') },
    { path: '/campanha/:id', name: 'campaign', component: () =>
import('../views/CampaignView.vue') },
    { path: '/cadastrar',    name: 'register', component: () =>
import('../views/RegisterView.vue') }
  ]
});
```

4.3. Parâmetros dinâmicos e sua extração

Parâmetro de rota `:id` (Tela 2). O trecho `:id` do caminho `/campanha/:id` indica um segmento variável. Ao acessar `/campanha/4`, o Vue Router associa o valor `"4"` à chave `id`. Como as telas usam a Composition API, o parâmetro é lido pelo *composable* `useRoute()`, que devolve o mesmo objeto acessível como `$route` no template (`route.params.id` ≡ `$route.params.id`). O valor é então usado para montar as três requisições à API:

```
// src/views/CampaignView.vue
import { useRoute } from 'vue-router';

const route = useRoute();
const id = route.params.id;              // '/campanha/4'  →  id = '4'

onMounted(async () => {
  const [resCampaign, resDonates, resVolunteers] = await Promise.all([
    fetch(`${import.meta.env.VITE_API_URL}/campanhas/${id}`),
    fetch(`${import.meta.env.VITE_API_URL}/doacoes?campanhaId=${id}`),
    fetch(`${import.meta.env.VITE_API_URL}/voluntarios?campanhaId=${id}`)
  ]);
  // ...converte para JSON e preenche data, donates e volunteers
});
```

As três requisições são disparadas em paralelo com `Promise.all`, reduzindo o tempo de carregamento da tela. O valor de `id` chega como *string*; a conversão para número é feita no backend (`Number(id)`).

Parâmetro de consulta `?campanha=` (Tela 3). A Tela 3 é acessada de duas formas: sem parâmetro (`/cadastrar`), para cadastrar uma nova causa, ou com a campanha escolhida na query string (`/cadastrar?campanha=4`), para doar ou se voluntariar. O valor é lido em `route.query.campanha`:

```
// src/views/RegisterView.vue
onMounted(async () => {
  const campaign = route.query.campanha;            // '/cadastrar?campanha=4' → '4'
  if (!campaign) {
    optionSelect.value = 'register';                // sem campanha: formulário "Cadastrar
causa"
  } else {
    const res  = await fetch(`${import.meta.env.VITE_API_URL}/campanhas/${campaign}`);
    const data = await res.json();
    if (!res.ok) { message.value = data.erro; messageType.value = 'error'; return; } // 404
    campaignData.value = data;
    // campanhas de voluntariado abrem a aba "Voluntariado"; as demais, a aba "Doar"
    optionSelect.value = data.categoria === 'voluntariado' ? 'volunteering' : 'donate';
  }
});
```

Optou-se por *query string* na Tela 3 porque o identificador da campanha é opcional nessa rota (a mesma tela atende o cadastro de campanhas, que não depende de uma campanha existente). Já na Tela 2 o identificador é obrigatório e faz parte do caminho.

> **Alternativa via props:** o mesmo parâmetro poderia ser entregue ao componente como *prop*, declarando `{ path: '/campanha/:id', component: ..., props: true }` no roteador e `defineProps({ id: String })` na tela. O projeto adota `useRoute()` por ser mais direto para os dois tipos de parâmetro (rota e query).

4.4. Fluxo de navegação entre as telas

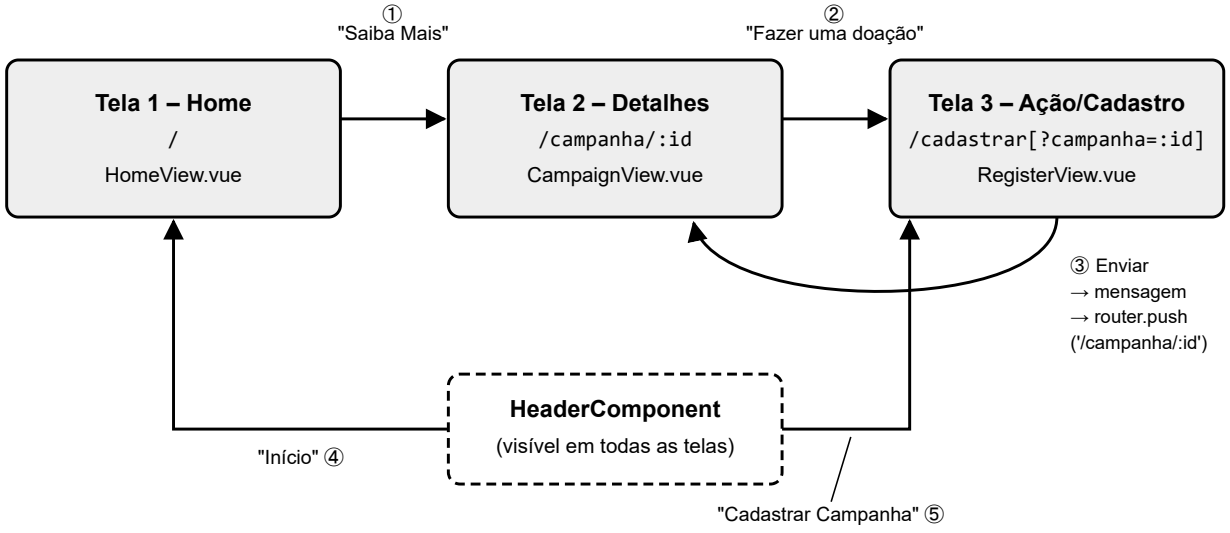


Figura 1 – Fluxo de transição entre as telas da SPA.

#	Origem → Destino	Elemento	Comportamento

①	Tela 1 → Tela 2	`<RouterLink :to="`/campanha/${id}`">` no botão “Saiba Mais” de cada `CardCampaignComponent`	O `id` da campanha chega ao card por *prop* (vinda do `v-for` da Home) e é interpolado na URL. O Vue Router troca a tela sem recarregar a página.
②	Tela 2 → Tela 3	`<RouterLink :to="`/cadastrar?campanha=${id}`">` no botão “Fazer uma doação”	Repassa o mesmo `id` lido de `route.params.id`, agora como query string. O botão só é exibido para campanhas que não estão encerradas.
③	Tela 3 → Tela 2 (pós-submissão)	Botão “Enviar” → `newDonate()`, `newVolunteering()` ou `newRegister()`	Em caso de sucesso, exibe a mensagem retornada pela API (ex.: “Doação registrada com sucesso”), limpa os campos e, após 1,5 s, redireciona com `router.push(`/campanha/${id}`)` para os detalhes da campanha, onde a nova doação/inscrição já aparece. No cadastro de campanha, o `id` usado é o da campanha recém-criada (retornado no corpo do `201`). Em caso de erro, a tela permanece em `/cadastrar` e exibe o campo `erro` da resposta (RF08).
④	Qualquer tela → Tela 1	`<RouterLink to="/">` “Início” no cabeçalho	Volta à listagem; os dados são buscados novamente no `onMounted`, refletindo doações recém-registradas.
⑤	Qualquer tela → Tela 3	Link “Cadastrar Campanha” no cabeçalho (`/cadastrar`)	Abre a Tela 3 sem query string, já com a aba “Cadastrar causa” selecionada (perfil Administrador).

4.5. Componentes reutilizáveis

Componente	**Usado em**	**Responsabilidade / props**
HeaderComponent	App.vue	Logo e menu de navegação (Início, Cadastrar Campanha).
FooterComponent	App.vue	Rodapé com a identidade da ONG.
HeroComponent	HomeView	Banner inicial com chamada e botão “Ver campanhas” (âncora `#campaigns`).
CardCampaignComponent	HomeView	Card de campanha. Props: `id, image, category, title, description, value, meta, volunteers, status`. Calcula o percentual da meta (`100 × value ÷ meta`) e resume a descrição em 100 caracteres.
ButtonComponent	Todas as telas	Botão padrão. Props: `text, icon, type`.

ToDoComponent / CardTodoComponent	Todas as telas	Seção “O que você pode fazer” (doar, ser voluntário, cadastrar causa).
InfoComponent	Todas as telas	Seção “Por que doar e se voluntariar?”.

4.6. Esboço das telas (wireframes de baixa fidelidade)

Os esboços abaixo representam a disposição dos elementos de cada tela. Áreas hachuradas indicam imagens; números em círculo indicam os elementos interativos descritos nas legendas.

Tela 1 – Visão Geral / Dashboard (`/`)

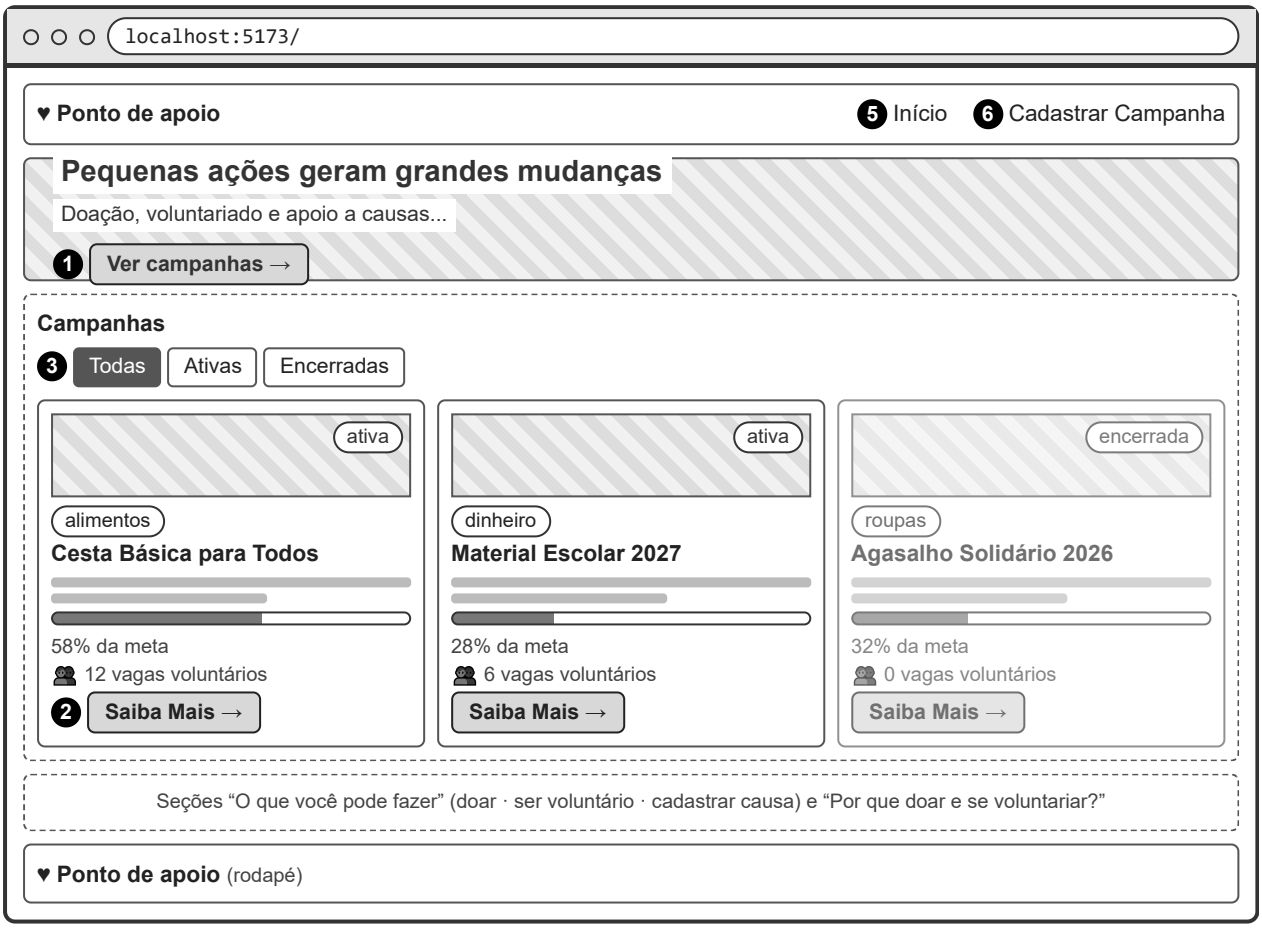


Figura 2 – Wireframe da Tela 1 (HomeView).

1	Âncora para a seção de campanhas na própria página.
2	“Saiba Mais”: navega para `/campanha/:id` com o id do card (fluxo ①).
3	Filtro por situação: refaz a busca com `GET /api/campanhas?status=ativa|encerrada` (ou sem filtro em “Todas”) (RF02).
5 / 6	Menu do cabeçalho: `/` e `/cadastrar` (fluxos ④ e ⑤).

Dados: no `onMounted`, a tela chama `GET /api/campanhas` e renderiza um `CardCampaignComponent` por campanha com `v-for` (`:key="campaign.id"`). Cada card mostra nome, categoria, situação e progresso da arrecadação (RF01). Os botões de filtro alteram a variável reativa `statusFilter` e chamam novamente a função `getCampaigns()`, que monta a URL com `?status=` (RF02). Quando o filtro não retorna resultados, é exibida a mensagem “Nenhuma campanha encontrada”.

Tela 2 – Detalhes e Acompanhamento (`/campanha/:id`)

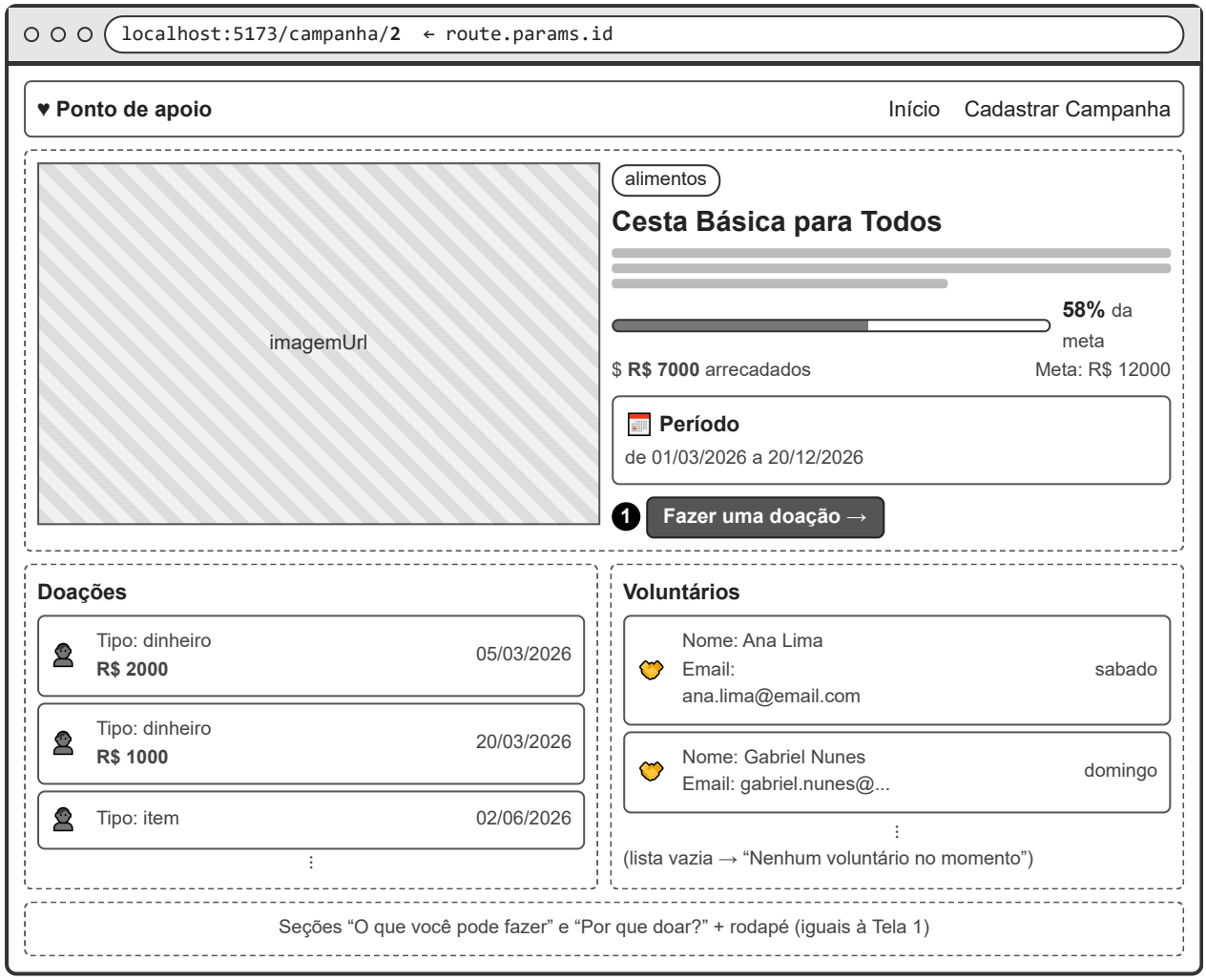


Figura 3 – Wireframe da Tela 2 (CampaignView).

1	“Fazer uma doação”: navega para `/cadastrar?campanha=:id` (fluxo ②). Oculto quando a campanha está encerrada (RN01).

Dados: a tela extrai o `id` de `route.params.id` e busca em paralelo `GET /api/campanhas/:id` (detalhes – RF03), `GET /api/doacoes?campanhaId=:id` (doações – RF04) e `GET /api/voluntarios?campanhaId=:id` (inscritos). Se a API responder `404`, a tela exibe “Campanha não encontrada” com um botão para voltar à listagem (RN09). O percentual da barra de progresso é calculado como `valorArrecadado ÷ metaValor × 100`, e as datas no formato `AAAA-MM-DD` são exibidas como `DD/MM/AAAA`.

Tela 3 – Ação / Cadastro (`/cadastrar`)

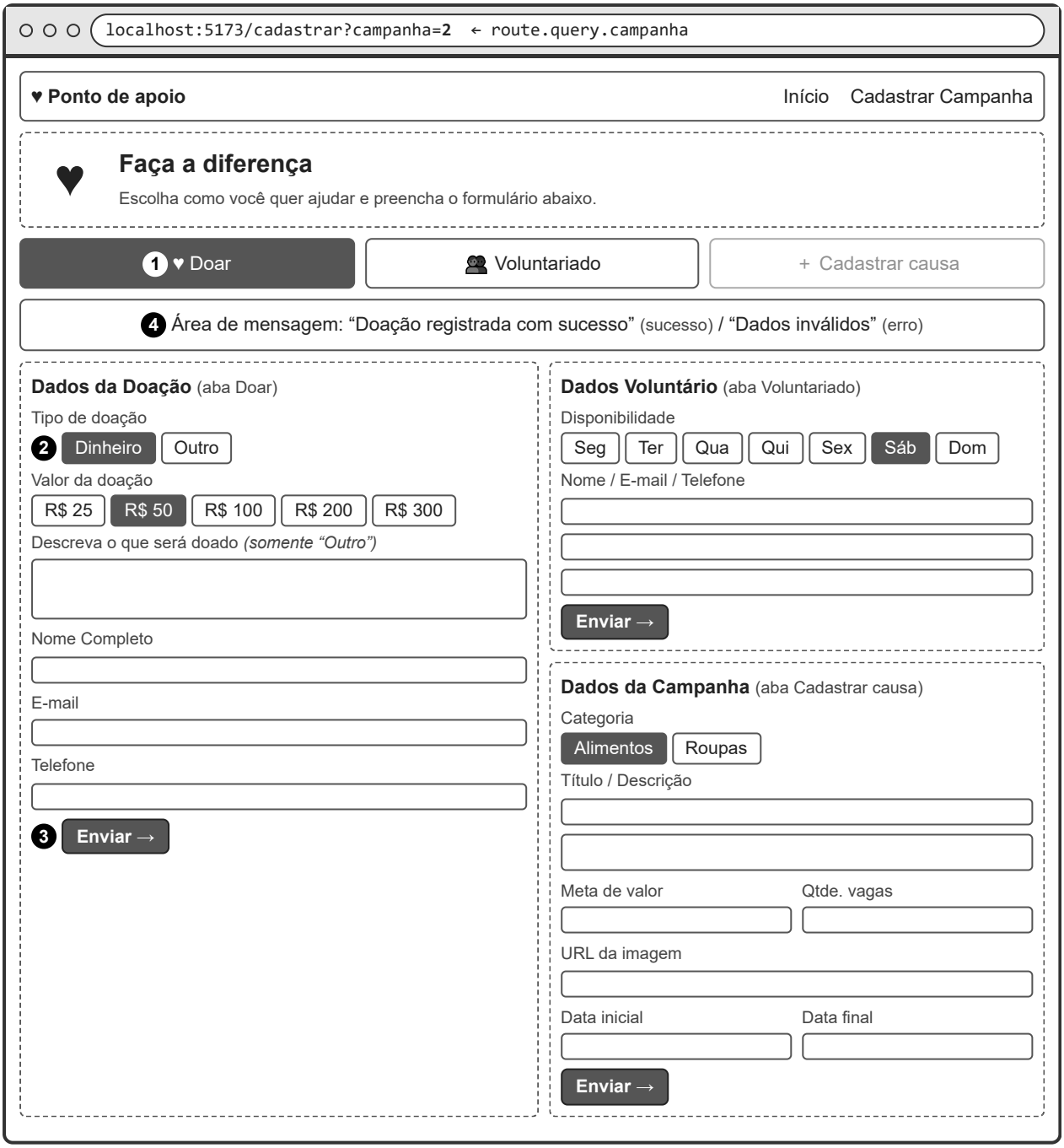


Figura 4 – Wireframe da Tela 3 (RegisterView) com as três variações de formulário.

1	Abas de ação. Com `?campanha=` a aba é escolhida pela categoria da campanha e “Cadastrar causa” fica desabilitada; sem o parâmetro só “Cadastrar causa” fica disponível. Se a campanha estiver encerrada, os formulários são ocultados e é exibido um aviso (RN01).
2	Tipo da doação: “Dinheiro” exige um valor (RN02); “Outro” exige a descrição do item (RN03).
3	“Enviar”: valida os campos obrigatórios no cliente e envia `POST /api/doacoes` , `POST /api/voluntarios` ou `POST /api/campanhas` ; após o sucesso redireciona para `/campanha/:id` (fluxo ③).

4	Mensagem de confirmação ou de erro devolvida pela API (RF08).

Mapeamento dos campos do formulário para o corpo da requisição: a doação converte o tipo `money` / `other` do formulário para `dinheiro` / `item`; o cadastro de campanha converte as datas digitadas em `DD/MM/AAAA` para `AAAA-MM-DD` antes do envio. Os payloads completos estão na seção 5.3.

5. Especificação da API REST (Node.js + Express.js)

O backend expõe uma API REST em `http://localhost:3000/api`, que atende às três telas descritas na seção 4. Toda comunicação é feita em JSON (RNF05).

5.1. Arquitetura do servidor

O servidor (`backend/src/server.js`) usa Express 5 em módulos ES e aplica, nesta ordem:

- `cors()` – libera o acesso do frontend (porta 5173) à API (porta 3000) (RNF04);
- `express.json()` – converte o corpo das requisições em objeto JavaScript;
- `app.use('/api', routes)` – registra as rotas com o prefixo `/api`;
- `notFound` – responde 404 para rotas inexistentes;
- `errorHandler` – centraliza o tratamento de erros (400, 404, 409, 500).

O código segue uma organização em camadas, em que cada camada tem uma única responsabilidade:

Camada	Arquivos	Responsabilidade
Rotas	`routes/index.js`	Associa verbo HTTP + URL a uma função do controller.
Controllers	`controllers/campanhaController.js`, `doacaoController.js`, `voluntarioController.js`	Lê `req.params`, `req.query` e `req.body`, chama o service e define o status HTTP da resposta. Erros são repassados com `next(err)`.
Services	`services/campanhaService.js`, `doacaoService.js`, `voluntarioService.js`	Validações e regras de negócio (RN01–RN08). Lançam `AppError(mensagem, status, detalhes)` quando uma regra é violada.
Dados	`data/database.js`	Arrays em memória: `campanhas`, `doadores`, `doacoes`, `voluntarios` (RNF03).
Middlewares / utils	`middlewares/errorHandler.js`, `utils/AppError.js`	Padronizam as respostas de erro.

5.2. Mapeamento de endpoints

Verbo	URL	Parâmetros	Descrição	Tela	Status
`GET`	`/api/campanhas`	query: `status`, `categoria` (opcionais)	Lista as campanhas, com filtro opcional por situação e categoria (RF01, RF02).	1	200, 500
`GET`	`/api/campanhas/:id`	rota: `id`	Retorna uma campanha pelo identificador (RF03, RN09).	2, 3	200, 404, 500
`POST`	`/api/campanhas`	body JSON	Cadastra uma nova campanha (RF07, RN08).	3	201, 400, 500
`GET`	`/api/doacoes`	query: `campanhaId` (opcional)	Lista as doações, todas ou de uma campanha (RF04).	2	200, 500
`POST`	`/api/doacoes`	body JSON	Registra uma doação em dinheiro ou item (RF05, RN01–RN05).	3	201, 400, 404, 500
`GET`	`/api/voluntarios`	query: `campanhaId` (opcional)	Lista os voluntários, todos ou de uma campanha.	2	200, 500
`POST`	`/api/voluntarios`	body JSON	Inscreve um voluntário em uma campanha (RF06, RN01, RN04, RN06, RN07).	3	201, 400, 404, 409, 500

5.3. Estrutura dos payloads (JSON)

`GET` `/api/campanhas?status=ativa`

Retorna um array de campanhas. Sem parâmetros, retorna todas; `status` aceita `ativa` ou `encerrada` e `categoria` aceita `roupas`, `alimentos`, `dinheiro` ou `voluntariado`.

Resposta 200 OK

```
[
  {
    "id": 2,
    "titulo": "Cesta Básica para Todos",
    "descricao": "Montagem e distribuição de cestas básicas mensais para 80 famílias
cadastradas.",
    "categoria": "alimentos",
    "metaValor": 12000,
    "valorArrecadado": 7000,
    "dataInicio": "2026-03-01",
    "dataFim": "2026-12-20",
    "status": "ativa",
    "vagasVoluntarios": 12,
    "imagemUrl": "https://images.unsplash.com/photo-1593113646773-028c64a8f1b8?
w=600&h=300&fit=crop"
  },
  { "id": 4, "titulo": "Natal Sem Fome", "categoria": "alimentos", "status": "ativa", "...":
"..." }
]
```

`GET` /api/campanhas/:id

Requisição

```
GET /api/campanhas/2
```

Resposta 200 OK

```
{
  "id": 2,
  "titulo": "Cesta Básica para Todos",
  "descricao": "Montagem e distribuição de cestas básicas mensais para 80 famílias
cadastradas.",
  "categoria": "alimentos",
  "metaValor": 12000,
  "valorArrecadado": 7000,
  "dataInicio": "2026-03-01",
  "dataFim": "2026-12-20",
  "status": "ativa",
  "vagasVoluntarios": 12,
  "imagemUrl": "https://images.unsplash.com/photo-1593113646773-028c64a8f1b8?
w=600&h=300&fit=crop"
}
```

Resposta 404 Not Found (campanha inexistente – RN09)

```
{ "erro": "Campanha não encontrada", "detalhes": null }
```

`POST` `/api/campanhas`

Campos obrigatórios: `titulo` (mín. 3 caracteres), `descricao` e `metaValor` (> 0). O servidor gera o `id`, define `status = "ativa"`, `valorArrecadado = 0` e `dataInicio` com a data atual (RN08).

Corpo da requisição

```
{
  "titulo": "Inverno Quentinho",
  "descricao": "Arrecadação de roupas de inverno para famílias atendidas pela ONG.",
  "categoria": "roupas",
  "metaValor": 3000,
  "dataFim": "2027-07-31",
  "vagasVoluntarios": 5,
  "imagemUrl": "https://images.unsplash.com/photo-1517677208171-0bc6725a3e60"
}
```

Resposta 201 Created

```
{
  "id": 11,
  "titulo": "Inverno Quentinho",
  "descricao": "Arrecadação de roupas de inverno para famílias atendidas pela ONG.",
  "categoria": "roupas",
  "metaValor": 3000,
  "valorArrecadado": 0,
  "dataInicio": "2026-10-27",
  "dataFim": "2027-07-31",
  "status": "ativa",
  "vagasVoluntarios": 5,
  "imagemUrl": "https://images.unsplash.com/photo-1517677208171-0bc6725a3e60"
}
```

Resposta 400 Bad Request (validação)

```
{
  "erro": "Dados inválidos",
  "detalhes": [
    "Título deve ter ao menos 3 caracteres",
    "Descrição é obrigatória",
    "O valor de meta deve ser maior que zero"
  ]
}
```

`GET` `/api/doacoes?campanhaId=4`

Resposta 200 OK

```
[
  { "id": 21, "campanhaId": 4, "doadorId": 3,  "tipo": "dinheiro", "valor": 300, "descricao": null,           "data": "2026-10-16" },
  { "id": 25, "campanhaId": 4, "doadorId": 15, "tipo": "item",     "valor": 0,   "descricao": "30 panetones", "data": "2026-10-24" }
]
```

`POST` `/api/doacoes`

O campo `tipo` deve ser `"dinheiro"` (exige `valor` > 0, que é somado ao `valorArrecadado` da campanha) ou `"item"` (exige `descricao` e grava `valor = 0`). O doador é localizado pelo e-mail; se não existir, é cadastrado automaticamente (RN05).

Corpo da requisição – doação em dinheiro

```
{
  "campanhaId": 2,
  "nome": "Maria Souza",
  "email": "maria.souza@email.com",
  "telefone": "47999991111",
  "tipo": "dinheiro",
  "valor": 50,
  "descricao": null
}
```

Corpo da requisição – doação de item

```
{
  "campanhaId": 2,
  "nome": "Mercado Bom Preço",
  "email": "contato@bompreco.com.br",
  "telefone": "4734335566",
  "tipo": "item",
  "valor": 0,
  "descricao": "15 kg de arroz"
}
```

Resposta 201 Created

```
{
  "mensagem": "Doação registrada com sucesso",
  "doacao": {
    "id": 60,
    "campanhaId": 2,
    "doadorId": 1,
    "tipo": "dinheiro",
    "valor": 50,
    "descricao": null,
    "data": "2026-10-27"
  }
}
```

Resposta 400 Bad Request (validação – RN02, RN03, RN04)

```
{
  "erro": "Dados inválidos",
  "detalhes": [ "Nome deve ter ao menos 3 caracteres", "Email inválido", "Valor inválido para doação em dinheiro" ]
}
```

Resposta 400 Bad Request (campanha encerrada – RN01)

```
{ "erro": "Campanha não está ativa para doação", "detalhes": null }
```

Resposta 404 Not Found (campanha inexistente – RN09)

```
{ "erro": "Campanha não encontrada", "detalhes": null }
```

`GET` /api/voluntarios?campanhaId=2

Resposta 200 OK

```
[
  { "id": 1, "campanhaId": 2, "nome": "Ana Lima",      "email": "ana.lima@email.com", "telefone": "(47) 97777-3333", "disponibilidade": "sabado" },
  { "id": 2, "campanhaId": 2, "nome": "Gabriel Nunes", "email": "gabriel.nunes@email.com", "telefone": "(47) 97111-0001", "disponibilidade": "domingo" }
]
```

`POST` /api/voluntarios

Cada inscrição aceita decrementa `vagasVoluntarios` da campanha em 1 (RN06).
`disponibilidade` aceita um dia da semana: `segunda`, `terca`, `quarta`, `quinta`, `sexta`, `sabado` ou `domingo`.

Corpo da requisição

```
{
  "campanhaId": 8,
  "nome": "Carlos Eduardo",
  "email": "carlos.eduardo@email.com",
  "telefone": "47991234567",
  "disponibilidade": "sabado"
}
```

Resposta 201 Created

```
{
  "mensagem": "Inscrição realizada com sucesso",
  "voluntario": {
    "id": 25,
    "campanhaId": 8,
    "nome": "Carlos Eduardo",
    "email": "carlos.eduardo@email.com",
    "telefone": "47991234567",
    "disponibilidade": "sabado"
  }
}
```

Respostas de erro

Status	Situação	Corpo
400	Nome/e-mail inválidos (RN04)	`{ "erro": "Dados inválidos", "detalhes": ["Email inválido"] }`
400	Campanha encerrada (RN01)	`{ "erro": "Campanha encerrada", "detalhes": null }`
400	Vagas esgotadas (RN06)	`{ "erro": "Não há vagas de voluntariado", "detalhes": null }`
404	Campanha inexistente (RN09)	`{ "erro": "Campanha não encontrada", "detalhes": null }`
409	E-mail já inscrito na campanha (RN07)	`{ "erro": "Voluntário já inscrito nesta campanha", "detalhes": null }`

5.4. Padrão de erros e códigos HTTP

Os services lançam instâncias de `AppError`, que carregam a mensagem, o código HTTP e uma lista opcional de detalhes. O middleware `errorHandler` transforma qualquer erro em uma resposta JSON com o formato `{ "erro": string, "detalhes": string[] | null }`:

```
// utils/AppError.js
class AppError extends Error {
  constructor(message, statusCode = 400, detalhes = null) { ... }
}

// middlewares/errorHandler.js
if (err instanceof AppError)
  return res.status(err.statusCode).json({ erro: err.message, detalhes: err.detalhes });
if (err.type === 'entity.parse.failed')
  return res.status(400).json({ erro: 'JSON inválido no corpo da requisição' });
res.status(500).json({ erro: 'Erro interno do servidor' });
```

Código	Significado	Quando é usado
200 OK	Sucesso	Consultas (`GET`) concluídas.
201 Created	Recurso criado	Campanha, doação ou inscrição registrada (`POST`).
400 Bad Request	Requisição inválida	Falha de validação, campanha não ativa, vagas esgotadas ou JSON malformado.
404 Not Found	Não encontrado	Campanha inexistente ou rota inexistente (ex.: `{ "erro": "Rota GET /api/xyz não encontrada" }`).
409 Conflict	Conflito	Voluntário já inscrito na mesma campanha com o mesmo e-mail.
500 Internal Server Error	Erro inesperado	Qualquer exceção não prevista; o detalhe é registrado no console do servidor.

5.5. Fluxo de uma requisição na API

O diagrama abaixo resume como uma requisição sai do cliente (Vue.js), percorre as camadas do servidor Express e retorna ao cliente, tanto em caso de sucesso quanto de erro.

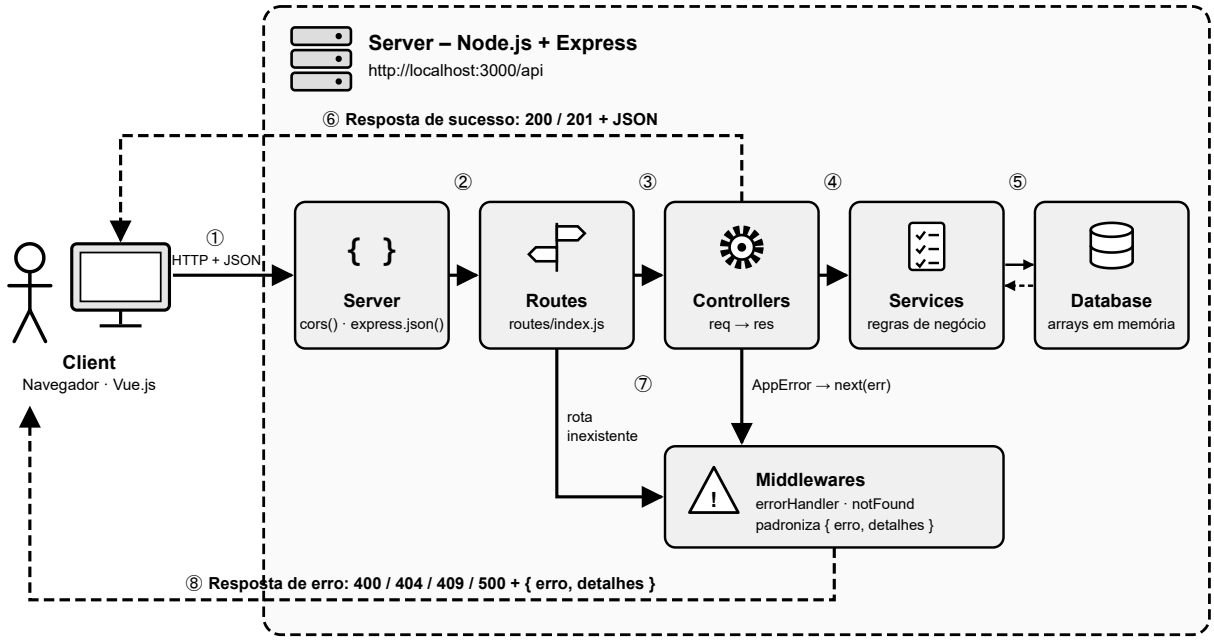


Figura 5 – Fluxo de uma requisição entre cliente e servidor (linha cheia = requisição; linha tracejada = resposta).

Fluxo	Descrição
①	O **Client** (tela Vue.js) envia a requisição com `fetch`, por exemplo `POST /api/doacoes` com o corpo em JSON.
②	O **Server** (`server.js`) libera o acesso com `cors()`, converte o corpo com `express.json()` e repassa para as rotas em `/api`.
③	As **Routes** identificam o verbo e o caminho e chamam a função do controller correspondente.
④	O **Controller** lê `req.body`, `req.params` ou `req.query` e chama o **Service**, que valida os dados e aplica as regras de negócio.
⑤	O **Service** lê e grava nos arrays do **Database** (`campanhas`, `doadores`, `doacoes`, `voluntarios`).
⑥	Com sucesso, o controller devolve ao cliente `res.status(200 | 201).json(...)`.
⑦	Se uma regra for violada, o service lança `AppError` e o controller o repassa com `next(err)`; se a rota não existir, a requisição cai no `notFound`.
⑧	Os **Middlewares** respondem ao cliente com o status do erro e o corpo padronizado `{ erro, detalhes }`.

6. Modelagem Simplificada de Dados

Nesta versão os dados ficam em arrays de objetos em memória, exportados por `backend/src/data/database.js` (RNF03). Os arrays são inicializados com dados fictícios (*mock data*) e alterados pelos services durante a execução; ao reiniciar o servidor, os dados voltam ao estado inicial.

6.1. Diagrama conceitual

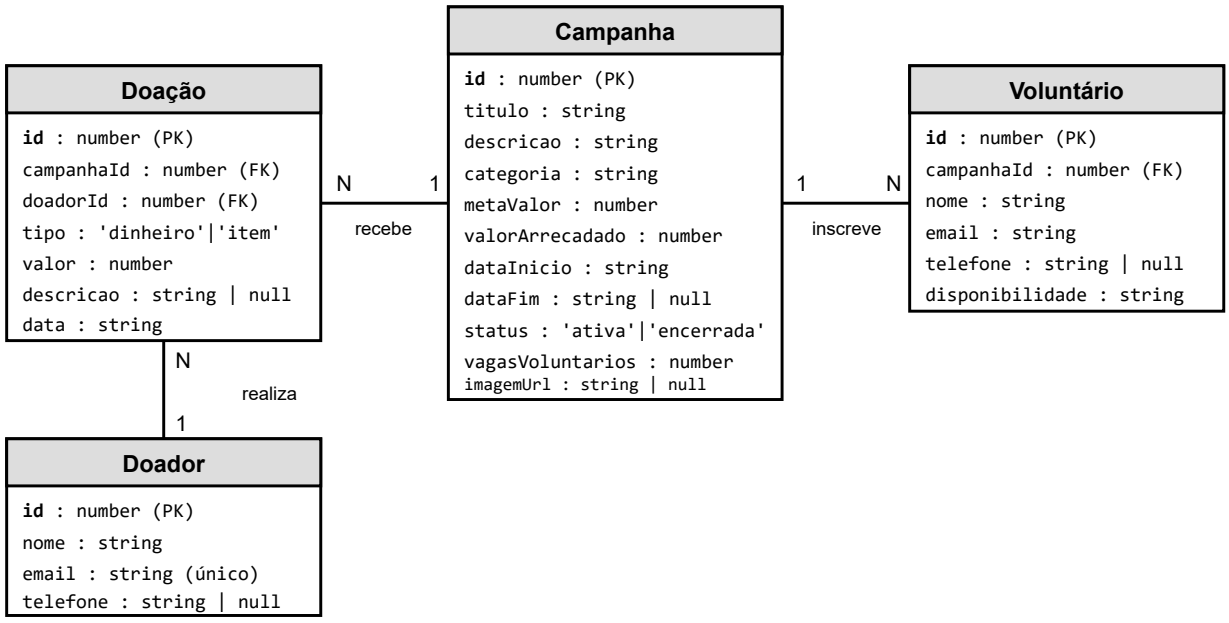


Figura 6 – Entidades e relacionamentos (PK = identificador, FK = referência a outra entidade).

- Uma **Campanha** recebe várias **Doações** e possui vários **Voluntários** inscritos.
- Um **Doador** pode realizar várias **Doações**; ele é identificado pelo e-mail e reutilizado em novas doações (RN05).
- O **Voluntário** é registrado por inscrição, ligado diretamente a uma campanha; o par `(campanhaId, email)` é único (RN07).

6.2. Estrutura dos objetos

Campanha – `const campanhas = [...]`

Campo	Tipo	Descrição / regra
`id`	number	Identificador gerado pelo servidor (maior id + 1).
`titulo`	string	Obrigatório, mínimo de 3 caracteres.
`descricao`	string	Obrigatória.
`categoria`	string	`roupas` | `alimentos` | `dinheiro` | `voluntariado`.

`metaValor`	number	Meta de arrecadação em reais; maior que zero.
`valorArrecadado`	number	Inicia em 0; soma das doações em dinheiro.
`dataInicio`	string	Data de criação (`AAAA-MM-DD`), definida pelo servidor.
`dataFim`	string | null	Data prevista de término (`AAAA-MM-DD`).
`status`	string	`ativa` | `encerrada` ; novas campanhas são criadas como `ativa` .
`vagasVoluntarios`	number	Vagas restantes; diminui a cada inscrição.
`imagemUrl`	string | null	Endereço da imagem exibida nos cards e nos detalhes.

Doador – `const doadores = [...]`

Campo	Tipo	Descrição / regra
`id`	number	Identificador gerado pelo servidor.
`nome`	string	Mínimo de 3 caracteres.
`email`	string	E-mail válido; chave de identificação do doador.
`telefone`	string | null	Contato opcional.

Doação – `const doacoes = [...]`

Campo	Tipo	Descrição / regra
`id`	number	Identificador gerado pelo servidor.
`campanhaId`	number	Campanha que recebeu a doação (deve estar ativa).
`doadorId`	number	Doador que realizou a doação.
`tipo`	string	`dinheiro` | `item` .
`valor`	number	Valor em reais (> 0) para dinheiro; 0 para item.
`descricao`	string | null	Obrigatória para item (ex.: “10 kg de arroz”).
`data`	string	Data do registro (`AAAA-MM-DD`).

Voluntário – `const voluntarios = [...]`

Campo	Tipo	Descrição / regra
`id`	number	Identificador gerado pelo servidor.
`campanhaId`	number	Campanha em que se inscreveu.
`nome`	string	Mínimo de 3 caracteres.
`email`	string	E-mail válido; não pode repetir na mesma campanha.

`telefone`	string | null	Contato.
`disponibilidade`	string | null	Dia da semana disponível (`segunda` … `domingo`).

6.3. Exemplo da base em memória (mock data)

```
// backend/src/data/database.js (trecho)
const campanhas = [
  {
    id: 2,
    titulo: 'Cesta Básica para Todos',
    descricao: 'Montagem e distribuição de cestas básicas mensais para 80 famílias
cadastradas.',
    categoria: 'alimentos',          // roupas | alimentos | dinheiro | voluntariado
    metaValor: 12000,
    valorArrecadado: 7000,
    dataInicio: '2026-03-01',
    dataFim: '2026-12-20',
    status: 'ativa',                 // ativa | encerrada
    vagasVoluntarios: 12,
    imagemUrl: 'https://images.unsplash.com/photo-1593113646773-028c64a8f1b8?...'
  },
  // ... 10 campanhas no total
];

const doadores = [
  { id: 1,  nome: 'Maria Souza',       email: 'maria.souza@email.com',   telefone: '(47)
99999-1111' },
  { id: 15, nome: 'Mercado Bom Preço', email: 'contato@bompreco.com.br', telefone: '(47)
3433-5566' }
  // ... 15 doadores (pessoas e empresas)
];

const doacoes = [
  { id: 8,  campanhaId: 2, doadorId: 15, tipo: 'dinheiro', valor: 2000, descricao: null,
data: '2026-03-05' },
  { id: 14, campanhaId: 2, doadorId: 2,  tipo: 'item',     valor: 0,    descricao: '10 kg de
arroz', data: '2026-06-02' }
  // ... 59 doações
];

const voluntarios = [
  { id: 1, campanhaId: 2, nome: 'Ana Lima', email: 'ana.lima@email.com',
    telefone: '(47) 97777-3333', disponibilidade: 'sabado' }
  // ... 24 inscrições
];

export { campanhas, doadores, doacoes, voluntarios };
```

6.4. Onde cada regra de negócio é aplicada

Regra	Arquivo / função	Implementação

RN01	`doacaoService.criar`, `voluntarioService.criar`	Verifica `campanha.status === 'ativa'` antes de registrar.
RN02	`doacaoService.validar` / `criar`	Exige `valor > 0` e executa `campanha.valorArrecadado += valor`.
RN03	`doacaoService.validar` / `criar`	Exige `descricao` e grava `valor = 0` para o tipo `item`.
RN04	`doacaoService.validar`, `voluntarioService.criar`	`nome.trim().length >= 3` e e-mail testado com a expressão `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`.
RN05	`doacaoService.obterDoador`	Procura o doador por e-mail em `doadores`; se não existir, cria um novo.
RN06	`voluntarioService.criar`	Rejeita quando `vagasVoluntarios <= 0`; após inscrever, executa `vagasVoluntarios -= 1`.
RN07	`voluntarioService.criar`	`voluntarios.some(v => v.campanhaId === campanha.id && v.email === dados.email)` → 409.
RN08	`campanhaService.criar`	Valida título, descrição e meta; cria com `status: 'ativa'` e `valorArrecadado: 0`.
RN09	`campanhaController.buscaPorId`, `doacaoService.criar`, `voluntarioService.criar`	Quando `buscarPorId` não encontra a campanha, lança `AppError('Campanha não encontrada', 404)`; no frontend, a Tela 2 exibe a mensagem correspondente.